

CHAPTER

Control II—Procedures and Environments

10.1	Procedure Definition and Activation	445
10.2	Procedure Semantics	447
10.3	Parameter-Passing Mechanisms	451
10.4	Procedure Environments, Activations, and Allocation	459
10.5	Dynamic Memory Management	473
10.6	Exception Handling and Environments	477
10.7	Case Study: Processing Parameter Modes in TinyAda	479

In the previous chapter, we discussed statement-level control structures and simple structured control mechanisms. In this chapter, we extend the study of blocks to the study of procedures and functions, which are blocks whose execution is deferred and whose interfaces are clearly specified.

Many languages make strong syntactic distinctions between procedures and functions. A case can be made for a significant semantic distinction as well. According to this argument, functions should (in an ideal world) produce a value only and have no side effects (thus sharing semantic properties with mathematical functions), while procedures produce no values and operate by producing side effects. Indeed, the distinction between procedures and functions is in essence the same as the difference between expressions and statements as discussed in the previous chapter. That is, procedure calls are statements, while function calls are expressions. However, most languages do not enforce semantic distinctions. Furthermore, functions can produce side effects as well as return values, whereas procedures may be written as functions in disguise—producing values through their parameters, while causing no other side effects. Thus, we shall not make a significant distinction between procedures and functions in this chapter, since in most languages their semantic properties are similar, even when their syntax is not.¹

Procedures were first introduced when memory was scarce as a way of splitting a program into small, separately compiled pieces. It is not uncommon, even today, to see FORTRAN or C code in which every procedure is compiled separately. However, in modern use, separate compilation is more closely associated with modules (see the next chapter), while procedures are used as a mechanism to abstract a group of related operations into a single operation that can be used repeatedly without repeating the code. Procedures can also represent recursive processes that are not as easily represented by other control structures. They can imitate or replace loop operations (as noted in Chapter 1, and of particular interest in functional languages).

While virtually all programming languages have some notion of procedure or function, the generality of procedures varies tremendously from language to language. This generality has a major impact on the structure and complexity of the runtime environment needed for proper execution of programs. Fortran77 has a relatively simple notion of procedure as a static entity without recursion. This led directly to the notion of a static environment in which all memory allocation is performed prior to the start of execution. The idea of a recursive, dynamic procedure, pioneered in Algol60, resulted in the stack-based environments common in most imperative and object-oriented languages today. As we saw in Chapters 1 and 3, LISP and other functional languages generalized the notion of function to the point that functions

¹C/C++ make virtually no syntactic difference between expressions and statements: Any expression can be turned into a statement by tacking on a semicolon (a so-called **expression statement**); the value of the expression is then simply thrown away.

are first-class data objects themselves—they can be dynamically created and used as values just like any other data structure. This led to even more dynamic environments that are not restricted to a stacklike structure. They also require dynamic memory management, including garbage collection. On the other hand, the basic notion of an **activation record** as the collection of data needed to maintain a single execution of a procedure, is common to virtually all runtime environments.

We begin this chapter with a review of the various syntactic ways of defining and calling procedures and functions. We continue with an investigation into the basic semantic properties of procedures, which represent a more complex notion of block structure than the one discussed in the previous chapter. We then study the major parameter-passing techniques, and the semantic behavior they generate. Next, we outline the structure of activation records and the three basic varieties of runtime environments, including mechanisms for implementing exception handling. We conclude with an overview of methods for maintaining dynamically allocated environments with garbage collection.

10.1 Procedure Definition and Activation

A **procedure** is a mechanism in a programming language for abstracting a group of actions or computations. The group of actions is called the **body** of the procedure, with the body of the procedure represented as a whole by the name of the procedure. A procedure is defined by providing a **specification** or **interface** and a body. The specification gives the name of the procedure, a list of the types and names of its **formal parameters**, and the type of its returned value, if any:

```
// C++ code
void intswap (int& x, int& y){ // specification
    int t = x;    // body
    x = y;        // body
    y = t;        // body
}
```

Procedure `intswap` swaps the values of its parameters `x` and `y`, using a local variable `t`.

In some languages, and in some situations, a procedure specification can be separated from its body, when the specification must be available in advance:

```
void intswap (int&, int&); // specification only
```

Note that this specification does not require the names of the formal parameters to be specified.² In C++ such a specification is called (confusingly) a *declaration*, while the complete definition (including the body) is called a *definition*. (In C, declarations are called *prototypes*.) Typically, even when a specification precedes a definition, it must be repeated with the body.

You **call**, or **activate**, a procedure by stating its name, together with **arguments** to the call, which correspond to its formal parameters:

```
intswap (a, b);
```

²They can of course be supplied if the programmer wishes, but they need not agree with names in other specifications or definitions.

A call to a procedure transfers control to the beginning of the body of the called procedure (the **callee**). When execution reaches the end of the body, control is returned to the **caller**. In some languages, control can be returned to the caller before reaching the end of the callee's body by using a **return-statement**:

```
// C++ code
void intswap (int& x, int& y){
    if (x == y) return;
    int t = x;
    x = y;
    y = t;
}
```

In some languages, such as FORTRAN, to call a procedure you must also include the keyword CALL, as in

```
CALL INTSWAP (A, B)
```

In FORTRAN, procedures are called **subroutines**.

As we have noted, some programming languages distinguish between procedures, which carry out their operations by changing their parameters or nonlocal variables, and **functions**, which appear in expressions and compute **returned values**. Functions may or may not also change their parameters and nonlocal variables. In C and C++, all procedures are implicitly functions; those that do not return values are declared void (such as the swap function above), while ordinary functions are declared to have the (return) type of the value that they return:

```
int max (int x, int y){
    return x > y ? x : y;
}
```

In Ada and FORTRAN, different keywords are used for procedures and functions:

```
-- Ada procedure
procedure swap ( x,y: in out integer) is
    t: integer;
begin
    if (x = y) then return;
    end if;
    t := x;
    x := y;
    y := t;
end swap;

-- Ada function
function max ( x,y: integer ) return integer is
begin
    if (x > y) then return x;
    else return y;
    end if;
end max;
```

Some languages allow only functions (that is, all procedures must return values). Functional languages in particular have this property; see Chapter 3.

Procedure and function declarations can be written in a form similar to constant declarations, using an equal sign, as in the following ML function declaration for a swap procedure:³

```
(* ML code *)
fun swap (x,y) =
  let val t = !x
  in
    x := !y;
    y := t
  end;
```

The use of an equal sign to declare procedures is justified, because a procedure declaration gives the procedure name a meaning that remains constant during the execution of the program. We could say that a procedure declaration creates a **constant procedure value** and associates a symbolic name—the name of the procedure—with that value.

A procedure communicates with the rest of the program through its parameters and also through **nonlocal references**, that is, references to variables declared outside of its own body. The **scope rules** that establish the meanings of nonlocal references were introduced in Chapter 7.

10.2 Procedure Semantics

From the point of view of semantics, a procedure is a block whose declaration is separated from its execution. In Chapter 7, we saw examples of blocks in C and Ada that are not procedure blocks; these blocks are always executed immediately when they are encountered. For example, in C, blocks A and B in the following code are executed as they are encountered:

```
A:
{ int x,y;
  ...
  x = y * 10;
B:
{ int i;
  i = x / 2;
  ...
} /* end B */
} /* end A */
```

In Chapter 7, you saw that the **environment** determines the allocation of memory and maintains the meaning of names during execution. In a block-structured language, when a block is encountered during execution, it causes the allocation of local variables and other objects corresponding to the declarations

³This procedure in ML has type 'a ref * 'a ref -> unit and is polymorphic; see Chapter 9. Note that its return type is unit, which is the equivalent of the C void type.

of the block. This memory allocated for the local objects of the block is called the **activation record** (or sometimes **stack frame**) of the block. The block is said to be **activated** as it executes under the bindings established by its activation record. As blocks are entered during execution, control passes from the activation of the surrounding block to the activation of the inner block. When the inner block exits, control passes back to the surrounding block. At this point, the activation record of the inner block is released, returning to the environment of the activation record of the surrounding block.

For example, in the preceding C code, during execution *x* and *y* are allocated in the activation record of block A, as shown in Figure 10.1.



Figure 10.1 The activation record for block A

When block B is entered, space is allocated in the activation record of B. See Figure 10.2.

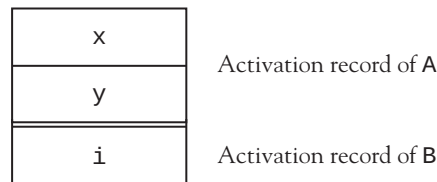


Figure 10.2 The activation records for blocks A and B

When B exits, the environment reverts to the activation record of A. Thus, the activation of B must retain some information about the activation from which it was entered.

In the preceding code, block B needs to access the variable *x* declared in block A. A reference to *x* inside B is a **nonlocal** reference, since *x* is not allocated in the activation record of B, but in the activation record of the surrounding block A. This, too, requires that B retain information about its surrounding activation.

Now consider what would happen if B were a procedure called from A instead of a block entered directly from A. Suppose, for the sake of a concrete example, that we have the following situation, which we give in C syntax:

```
(1) int x;
(2) void B(void)
(3) { int i;
(4)   i = x / 2;
(5)   ...
(6) } /* end B */
(7) void A(void)
(8) { int x, y;
```

(continues)

(continued)

```
(9)    ...
(10)   x = y * 10;
(11)   B();
(12) } /* end A */

(13) main()
(14) { A();
(15)   return 0;
(16) }
```

B is still entered from A (line 11), and the activation of B must retain some information about the activation of A so that control can return to A on exit from B, but there is now a difference in the way nonlocal references are resolved: Under the lexical scoping rule (see Chapter 7), the x in B (line 4) is the global x of the program (line 1),⁴ not the x declared in A (line 8). In terms of activation records, we have the picture shown in Figure 10.3.

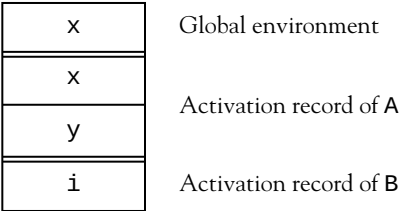


Figure 10.3 The activation records of the global environment, block A, and a procedure call of B

The activation of B must retain information about the global environment, since the nonlocal reference to x will be found there instead of in the activation record of A. This is because the global environment is the **defining environment** of B, while the activation record of A is the **calling environment** of B. (Sometimes the defining environment is called the **static environment** and the control environment the **dynamic environment**.) For blocks that are not procedures, the defining environment and the calling environment are always the same. By contrast, a procedure has different calling and defining environments. Indeed, a procedure can have any number of calling environments during which it will retain the same defining environment.

The actual structure of the environment that keeps track of defining and calling environments will be discussed in the next section. In this section, we are interested in the way an activation of a block **communicates** with the rest of the program.

Clearly, a nonprocedure block communicates with its surrounding block via nonlocal references: Lexical scoping allows it access to all the variables in the surrounding block that are not redeclared in its own declarations. By contrast, under lexical scoping, a procedure block can communicate only with its

⁴Technically, x is called an **external** variable, but, by the scope rules of C, such a variable has scope extending from its declaration to the end of the file in which it is declared; thus (ignoring separate compilation issues), it is global to the program.

defining block via references to nonlocal variables. It has no way of directly accessing the variables in its calling environment. In the sample C code, procedure B cannot directly access the local variable *x* of procedure A. Nevertheless, it may need the value of *x* in A to perform its computations.

The method of communication of a procedure with its calling environment is through **parameters**. A **parameter list** is declared along with the definition of the procedure, as in the *max* procedure of the previous section:

```
int max (int x, int y){
    return x > y ? x : y;
}
```

Here, *x* and *y* are parameters to the *max* function. They do not take on any value until *max* is called, when they are replaced by the arguments from the calling environment, as in:

```
z = max(a+b, 10);
```

In this call to *max*, the parameter *x* is replaced by the argument *a + b*, and the parameter *y* is replaced by the argument 10. To emphasize the fact that parameters have no value until they are replaced by arguments, parameters are sometimes called **formal parameters**, while arguments are called **actual parameters**.

A strong case can be made that procedures should communicate with the rest of the program using *only* the parameters—that is, a procedure or function should *never* use or change a nonlocal variable. After all, such use or change represents a dependency (in the case of a use) or a side effect (in the case of a change) that is invisible from a reading of the procedure specification—it can only be determined from examining the body of the procedure (which could be very long, or even missing in the case of a library procedure). While this rule is generally a good one with regard to *variables* (e.g., a global variable should still be a parameter if it is changed), it is too much to expect with regard to *functions* and *constants*. In general, the body of a procedure will need to call other procedures in the program and library procedures, and may also need to use constants (such as *PI*). These are all nonlocal references, but we would not want to clutter up the parameter list with all such references (particularly since they are *all* essentially constants). Thus, nonlocal *uses* cannot be avoided.

Nevertheless, some procedures may depend only on parameters and fixed language features. These are said to be in **closed form**, since they contain no nonlocal dependencies. An example is the *max* function above, which has no dependencies on nonlocal definitions—it depends entirely on fixed language features and its parameters.

On the other hand, a polymorphic definition of the same function in C++ (using templates):

```
template <typename T>
T max (T x, T y)
{ return x > y ? x : y ; }
```

is *not* in closed form, since the “>” operator can be overloaded for non-built-in types. Thus, to write this function in closed form, we would have to include the “>” operation in the parameter list:

```
template <typename T>
T max (T x, T y, bool (*gt)(T,T))
{ return gt(x,y) ? x : y ;
}
```

(These examples were discussed in detail in Section 8.9.)

If we choose not to do this, then the semantics of this function can only be determined relative to its enclosing environment (its environment of definition assuming static scoping). The code of this function together with a representation of its defining environment is called a **closure**, because it can be used to resolve all outstanding nonlocal references relative to the body of the function. It is one of the major tasks of a runtime environment to compute closures for all functions in cases where they are needed.⁵

An additional issue affecting the semantics of a procedure is the nature of the correspondence between the parameters and the arguments during a call. Earlier we stated that parameters are replaced by the arguments during a call, but the nature of this replacement was not specified. In fact, this association is a binding that can be interpreted in many different ways. This interpretation is called the **parameter-passing mechanism**. In the next section, we discuss this topic in some detail.

10.3 Parameter-Passing Mechanisms

As noted at the end of the previous section, the nature of the bindings of arguments to parameters has a significant effect on the semantics of procedure calls. Languages differ substantially as to the kinds of parameter-passing mechanisms available and the range of permissible implementation effects that may occur. (You have already seen one of these implementation effects in Chapter 9, namely the effect different evaluation orders of the arguments can have in the presence of side effects.) Some languages (C, Java, functional languages) offer only one basic kind of parameter-passing mechanism, while others (C++) may offer two or more. In languages with one mechanism, it is also often possible to imitate other mechanisms by using indirection or other language features.

In this section, we will discuss four important parameter-passing mechanisms: pass by value, pass by reference, pass by value-result, and pass by name.⁶ Some variations on these will be discussed in the exercises.

10.3.1 Pass by Value

In **pass by value**, by far the most common mechanism for parameter passing, the arguments are expressions that are evaluated at the time of the call, with the arguments' values becoming the values of the parameters during the execution of the procedure. In the simplest form of pass by value, value parameters behave as constant values during the execution of the procedure. Thus, we can think of pass by value as a process in which all the parameters in the body of the procedure are replaced by the corresponding argument values. For instance, we can think of the call `max (10, 2 + 3)` of the preceding `max` function as executing the body of `max` with `x` replaced by `10` and `y` replaced by `5`:

```
10 > 5 ? 10 : 5
```

⁵In this particular example, the reference to ">" can be resolved at compile time, and no closure needs to be computed, as we will see later in this chapter.

⁶Parameter-passing mechanisms are often referred to using the words "call by" rather than "pass by," as in "call by value," "call by reference," etc. We prefer "pass by" and use it consistently in this text.

This form of pass by value is usually the only parameter-passing mechanism in functional languages. Pass by value is also the default mechanism in C++ and Pascal, and is essentially the only parameter-passing mechanism in C and Java. However, in these languages a slightly different interpretation of pass by value is used: The parameters are viewed as local variables of the procedure, with initial values given by the values of the arguments in the call. Thus, in C, Java, and Pascal, value parameters may be assigned, just as with local variables (but cause no changes outside the procedure), while Ada in parameters may not be the targets of assignment statements (see the discussion of Ada parameters in Section 10.3.5).

Note that pass by value does not imply that changes cannot occur outside the procedure through the use of parameters. If the parameter has a pointer or reference type, then the value is an address and can be used to change memory outside the procedure. For example, the following C function definitely changes the value of the integer pointed to by the parameter `p`:

```
void init_p (int* p)
{ *p = 0; }
```

On the other hand, directly assigning to `p` does *not* change the argument outside the procedure:

```
void init_ptr (int* p)
{ p = (int*) malloc(sizeof(int)); /* error - has no effect! */
}
```

In addition, in some languages certain values are implicitly pointers or references. For example, in C, arrays are implicitly pointers (to the first location of the array); this means an array value parameter can always be used to change the values stored in the array:

```
void init_p_0 (int p[])
{ p[0] = 0; }
```

In Java, too, objects are implicitly pointers, so any object parameter can be used to change its data:

```
void append_1 (Vector v)
// adds an element to Vector v
{ v.addElement( new Integer(1)); }
```

However, as in C, direct assignments to parameters do not work either:

```
void make_new (Vector v)
{ v = new Vector(); /* error - has no effect! */ }
```

10.3.2 Pass by Reference

In pass by reference, an argument must be something like a variable with an allocated address. Instead of passing the value of the variable, pass by reference passes the location of the variable, so that the parameter becomes an **alias** for the argument, and any changes made to the parameter occur to the argument as well. In FORTRAN, pass by reference is the only parameter-passing mechanism. In C++ and Pascal, pass by reference (instead of the default pass by value) can be specified

by using an ampersand (&) after the data type in C++, and a `var` keyword before the variable name in Pascal:

```
void inc(int& x) // C++
{ x++ ; }

procedure inc(var x: integer); (* Pascal *)
begin
  x := x + 1;
end;
```

After a call to `inc(a)` the value of `a` has increased by 1, so that a side effect has occurred. Multiple aliasing is also possible, such as in the following example:

```
int a;

void yuck (int& x, int& y)
{ x = 2;
  y = 3;
  a = 4;
}
...
yuck (a, a);
```

Inside procedure `yuck` after the call, the identifiers `x`, `y`, and `a` all refer to the same variable, namely, the variable `a`.

As noted previously, C can achieve pass by reference by passing a reference or location explicitly as a pointer. C uses the operator `&` to indicate the location of a variable and the operator `*` to dereference a pointer. For example:

```
void inc (int* x) /*C imitation of pass by reference */
{ (*x)++; /* adds 1 to *x */ }
...
int a;
...
inc(&a); /* pass the address of a to the inc function */
```

This code has the same effect as the previous C++ or Pascal code for the `inc` procedure. Of course, there is the annoying necessity here of explicitly taking the address of the variable `a`, and then explicitly dereferencing it again in the body of `inc`.

A similar effect can be achieved in ML by using reference types to imitate pass by reference. (In ML, as in C, only pass by value is available.) For example:

```
fun inc (x: int ref) = x := !x + 1;
```

One additional issue that must be resolved in a language with pass by reference is the response of the language to reference arguments that are not variables and, thus, do not have known addresses within the runtime environment. For example, given the C++ code

```
void inc(int& x)
{ x++; }
...
inc(2); // ??
```

how should the compiler respond to the call `inc(2)`? In FORTRAN, in which pass by reference is the *only* available parameter mechanism, this is syntactically correct code. The response of the compiler is to create a temporary integer location, initialize it with the value 2, and then apply the `inc` function. This actually mimics pass by value, since the change to the temporary location does not affect the rest of the program. In C++ and Pascal, however, this is an error. In these languages, reference arguments *must* be l-values—that is, they must have known addresses.

10.3.3 Pass by Value-Result

Pass by value-result achieves a similar result to pass by reference, except that no actual alias is established. Instead, in pass by value-result, the value of the argument is copied and used in the procedure, and then the final value of the parameter is copied back out to the location of the argument when the procedure exits. Thus, this method is sometimes known as **copy-in**, **copy-out**, or **copy-restore**.

Pass by value-result is only distinguishable from pass by reference in the presence of aliasing. Thus, in the following code (written for convenience in C syntax):

```
void p(int x, int y)
{ x++;
  y++;
}

main()
{ int a = 1;
  p(a,a);
  ...
}
```

`a` has value 3 after `p` is called if pass by reference is used, while `a` has the value 2 if pass by value-result is used.

Issues left unspecified by this mechanism, and possibly differing in different languages or implementations, are the order in which results are copied back to the arguments and whether the locations of the arguments are calculated only on entry and stored or whether they are recalculated on exit.

A further option is that a language can offer a **pass by result** mechanism as well, in which there is no incoming value, but only an outgoing one.

10.3.4 Pass by Name and Delayed Evaluation

Pass by name is the term used for this mechanism when it was introduced in Algol60. At the time it was intended as a kind of advanced inlining process for procedures, so that the semantics of procedures could be described simply by a form of textual replacement, rather than an appeal to environments and closures. (See Exercise 10.14.) It turned out to be essentially equivalent to the normal order delayed evaluation described in the previous chapter. It also turned out to be difficult to implement, and to have complex interactions with other language constructs, particularly arrays and assignment. Thus, it was rarely implemented and was dropped in all Algol60 descendants (AlgolW, Algol68, C, Pascal, etc.). Advances in delayed evaluation in functional languages, particularly pure functional languages such as Haskell (where interactions with side effects are avoided), have increased interest in this mechanism, however, and it is worthwhile to understand it as a basis for other delayed evaluation mechanisms, particularly the more efficient **lazy evaluation** studied in Chapter 3.

The idea of pass by name is that the argument is not evaluated until its actual use as a parameter in the called procedure. Thus, the *name* of the argument, or its textual representation at the point of call, replaces the name of the parameter to which it corresponds. As an example, in the C code

```
void inc(int x)
{ x++; }
```

if a call such as `inc(a[i])` is made, the effect is of evaluating `a[i]++`. Thus, if `i` were to change before the use of `x` inside `inc`, the result would be different from either pass by reference or pass by value-result:

```
int i;
int a[10];

void inc(int x)
{ i++;
  x++;
}

main()
{ i = 1;
  a[1] = 1;
  a[2] = 2;
  inc(a[i]);
  return 0;
}
```

This code has the result of setting `a[2]` to 3 and leaving `a[1]` unchanged.

Pass by name can be interpreted as follows. The text of an argument at the point of call is viewed as a function in its own right, which is evaluated every time the corresponding parameter name is reached in the code of the called procedure. However, the argument will always be evaluated in the environment

of the caller, while the procedure will be executed in its defining environment. To see how this works, consider the example in Figure 10.4.

```
(1) #include <stdio.h>
(2) int i;

(3) int p(int y)
(4) { int j = y;
(5)   i++;
(6)   return j+y;
(7) }

(8) void q(void)
(9) { int j = 2;
(10)  i = 0;
(11)  printf("%d\n", p(i + j));
(12) }

(13) main()
(14) { q();
(15)   return 0;
(16) }
```

Figure 10.4 Pass by name example (in C syntax)

The argument `i + j` to the call to `p` from `q` is evaluated every time the parameter `y` is encountered inside `p`. The expression `i + j` is, however, evaluated as though it were still inside `q`, so on line 4 in `p` it produces the value 2. Then, on line 6, since `i` is now 1, it produces the value 3 (the `j` in the expression `i + j` is the `j` of `q`, so it hasn't changed, even though the `i` inside `p` has). Thus, if pass by name is used for the parameter `y` of `p` in the program, the program will print 5.

Historically, the interpretation of pass by name arguments as functions to be evaluated during the execution of the called procedure was expressed by referring to the arguments as **thunks**.⁷ For example, the above C code could actually imitate pass by name using a function, except for the fact that it uses the local definition of `j` inside `q`. If we make `j` global, then the following C code will actually print 5, just as if pass by name were used in the previous code:

```
#include <stdio.h>
int i, j;

int i_plus_j(void)
{ return i+j; }
```

(continues)

⁷Presumably, the image was of little machines that “thunked” into place each time they were needed.

(continued)

```
int p(int (*y)(void))
{ int j = y();
  i++;
  return j+y();
}

void q(void)
{ j = 2;
  i = 0;
  printf("%d\n", p(i_plus_j));
}

main()
{ q();
  return 0;
}
```

Pass by name is problematic when side effects are desired. Consider the `intswap` procedure we have discussed before:

```
void intswap (int x, int y)
{ int t = x;
  x = y;
  y = t;
}
```

Suppose that pass by name is used for the parameters `x` and `y` and that we call this procedure as follows,

```
intswap(i, a[i])
```

where `i` is an integer index and `a` is an array of integers. The problem with this call is that it will function as the following code:

```
t = i;
i = a[i];
a[i] = t;
```

Note that by the time the address of `a[i]` is computed in the third line, `i` has been assigned the value of `a[i]` in the previous line. This will not assign `t` to the array `a` subscripted at the original `i`, unless `i = a[i]`.

It is also possible to write bizarre (but possibly useful) code in similar circumstances. One of the earliest examples of this is called **Jensen's device** after its inventor J. Jensen. Jensen's device uses pass by name to apply an operation to an entire array, as in the following example, in C syntax:

```
int sum (int a, int index, int size)
{ int temp = 0;
  for (index = 0; index < size; index++) temp += a;
  return temp;
}
```

If `a` and `index` are pass by name parameters, then in the following code:

```
int x[10], i, xtotal;
...
xtotal = sum(x[i], i, 10);
```

the call to `sum` computes the sum of all the elements `x[0]` through `x[9]`.

10.3.5 Parameter-Passing Mechanism versus Parameter Specification

One can fault these descriptions of parameter-passing mechanisms in that they are tied closely to the internal mechanics of the code that is used to implement them. While one can give somewhat more theoretical semantic descriptions of these mechanisms, all of the questions of interpretation that we have discussed still arise in code that contains side effects. One language that tries to address this issue is Ada. Ada has two notions of parameter communication, `in` parameters and `out` parameters. Any parameter can be declared `in`, `out`, or `in out` (i.e., both). The meaning of these keywords is exactly what you would expect: An `in` parameter specifies that the parameter represents an incoming value only; an `out` parameter specifies an outgoing value only; and an `in out` parameter specifies both an incoming and an outgoing value. (The `in` parameter is the default, and the keyword `in` can be omitted in this case.)

Any parameter implementation whatsoever can be used to achieve these results, as long as the appropriate values are communicated properly (an `in` value on entry and an `out` value on exit). Ada also declares that any program that violates the protocols established by these parameter specifications is **erroneous**. For example, an `in` parameter cannot legally be assigned a new value by a procedure, and the value of an `out` parameter cannot be legally used by the procedure. Thus, pass by reference could be used for an `in` parameter as well as an `out` parameter, or pass by value could be used for `in` parameters and copy out for `out` parameters.

Fortunately, a translator can prevent many violations of these parameter specifications. In one case, an `in` parameter cannot be assigned to or otherwise have its value changed, because it should act like a constant. In another case, an `out` parameter can only be assigned to, because its value should never be used. Unfortunately, `in out` parameters cannot be checked with this specificity. Moreover, in the presence of other side effects, as we have seen, different implementations (reference and copy in-copy out, for example) may have different results. Ada still calls such programs erroneous, but translators cannot in general check whether a program is erroneous under these conditions. Thus, the outcome of this specification effort is somewhat less than what we might have hoped for.

10.3.6 Type Checking of Parameters

In strongly typed languages, procedure calls must be checked so that the arguments agree in type and number with the parameters of the procedure. This means, first of all, that procedures may not have a variable number of parameters and that rules must be stated for the type compatibility between parameters and arguments. In the case of pass by reference, parameters usually must have the same type, but in the case of pass by value, this can be relaxed to assignment compatibility (Chapter 8), and can allow conversions, as is done in C, C++, and Java. Ada, of course, does not allow conversions.

10.4 Procedure Environments, Activations, and Allocation

In this section, we give more detail on how information can be computed and maintained in an environment during procedure calls. We already saw in Chapter 7 and in Section 10.2 that the environment for a block-structured language with lexical scope can be maintained in a stack-based fashion, with an activation record created on the environment stack when a block is entered and released when the block is exited. We also saw how variables declared locally in the block are allocated space in this activation record.

In this section, we explore how this same structure can be extended to procedure activations, in which the defining environment and the calling environment differ. We also survey the kinds of information necessary to maintain this environment correctly. We have already noted that some notion of **closure** is necessary in this case to resolve nonlocal references. A clear understanding of this **execution model** is often necessary to fully understand the behavior of programs, since the semantics of procedure calls are embedded in this model.

We also want to show that a completely stack-based environment is no longer adequate to deal with procedure variables and the dynamic creation of procedures, and that languages with these facilities, particularly functional languages, are forced to use a more complex fully dynamic environment with garbage collection.

But, first, by way of contrast, we give a little more detail about the fully static environment of FORTRAN, which is quite simple, yet completely adequate for that language. We emphasize that the structures that we present here are only meant to illustrate the general scheme. Actual details implemented by various translators may differ substantially from the general outlines given here.

10.4.1 Fully Static Environments

In a language like Fortran77,⁸ all memory allocation can be performed at load time, and the locations of all variables are fixed for the duration of program execution. Function and procedure (or subroutine) definitions cannot be nested (that is, all procedures/functions are global, as in C),⁹ and recursion is not allowed (unlike C).¹⁰ Thus, all the information associated with a function or subroutine can be statically

⁸Fortran90 significantly extends the features of Fortran77, including allowing recursion and dynamically allocated variables.

⁹In Section 10.4.2, we will see why nested procedures present problems even for nonstatic environments.

¹⁰More precisely, recursive calls are permitted in Fortran77, but each new call overwrites the data of all previous calls, so recursion does not work. Procedures with this property are sometimes called **non-reentrant**.

allocated. Each procedure or function has a fixed activation record, which contains space for the local variables and parameters, and possibly the return address for proper return from calls. Global variables are defined by `COMMON` statements, and they are determined by pointers to a common area. Thus, the general form of the runtime environment for a FORTRAN program with subroutines `S1 . . . Sn` is as shown in Figure 10.5.

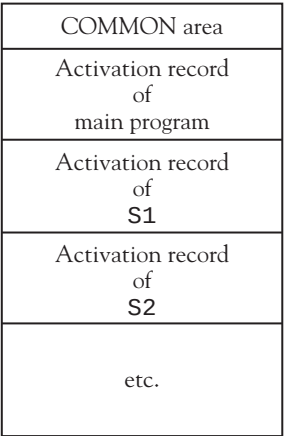


Figure 10.5 The runtime environment of a FORTRAN program with subprograms S1 and S2

Each activation record, moreover, is broken down into several areas, as shown in Figure 10.6.

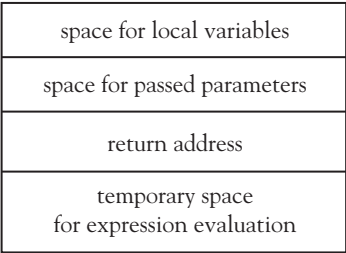


Figure 10.6 The component areas of an activation record

When a call to a procedure `S` occurs, the parameters are evaluated, and their locations (using pass by reference) are stored in the parameter space of the activation record of `S`. Then, the current instruction pointer is stored as the return address, and a jump is performed to the instruction pointer of `S`. When `S` exits, a jump is performed to the return address. If `S` is a function, special arrangements must be made for the returned value; often it is returned in a special register, or space can be reserved for it in the activation record of the function or the activation record of the caller. Note that all nonlocal references must be to the global `COMMON` area, so no closure is needed to resolve these references. Thus, no extra bookkeeping information is necessary in an activation record to handle nonlocal references.

As an example, consider the following FORTRAN program:

```

REAL TABLE (10), MAXVAL
READ *, TABLE (1), TABLE (2), TABLE (3)
CALL LRGST (TABLE, 3, MAXVAL)
PRINT *, MAXVAL
END

SUBROUTINE LRGST (A, SIZE, V)
INTEGER SIZE
REAL A (SIZE), V
INTEGER K
V = A (1)
DO 10 K = 1, SIZE
  IF (A( K) GT. V) V = A (K)
10 CONTINUE
RETURN
END

```

The environment of this program would look as shown in Figure 10.7, where we have added pointers indicating location references that exist immediately after the call to LRGST from the main program.

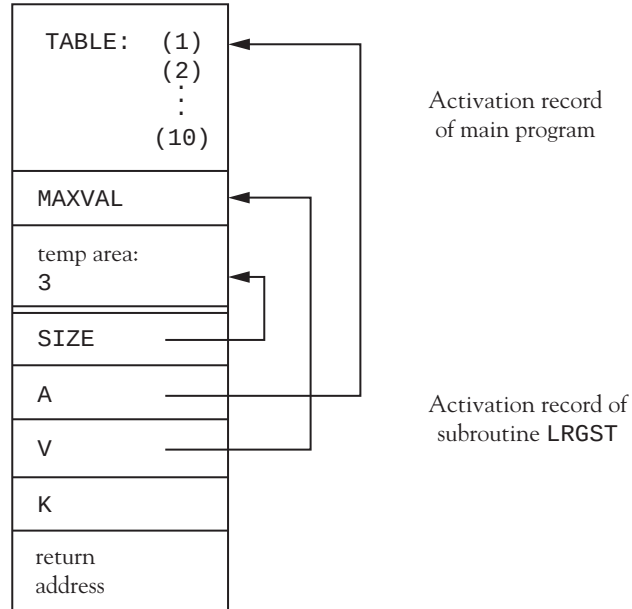


Figure 10.7 The runtime environment immediately after a call to the subroutine LRGST

10.4.2 Stack-Based Runtime Environments

In a block-structured language with recursion, such as C and other Algol-like languages, activations of procedure blocks cannot be allocated statically. Why? Because a procedure may be called again before its previous activation is exited, so a new activation must be created on each procedure entry. As you have seen, this can be done in a stack-based manner, with a new activation record created on the stack every time a block is entered and released on exit.

What information must be kept in the environment to manage a stack-based environment? As with the fully static environment of FORTRAN, an activation is necessary to allocate space for local variables, temporary space, and a return pointer. However, several additional pieces of information are required. First, a pointer to the current activation must be kept, since each procedure has no fixed location for its activation record, but the location of its activation record may vary as execution proceeds. This pointer to the current activation must be kept in a fixed location, usually a register, and it is called the **environment pointer** or **ep**, since it points to the current environment.

The second piece of information required in a stack-based environment is a pointer to the activation record of the block from which the current activation was entered. In the case of a procedure call, this is the activation of the caller. This piece of information is necessary because, when the current activation is exited, the current activation record needs to be removed (i.e., popped) from the stack of activation records. This means that the ep must be restored to point to the previous activation, a task that can be accomplished only if the old ep, which pointed to the previous activation record, is retained in the new activation record. This stored pointer to the previous activation record is called the **control link**, since it points to the activation record of the block from which control passed to the current block and to which control will return. Sometimes this control link is called the **dynamic link**.

A simple example is the following program, in C syntax:

```
void p(void)
{ ... }

void q(void)
{ ...
  p();
}

main()
{ q();
  ...
}
```

At the beginning of execution of the program, there is just an activation record for `main` and the ep points there (we ignore global data in this example). See Figure 10.8.

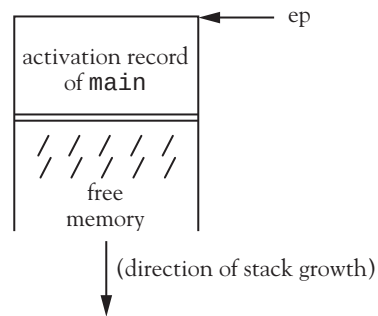


Figure 10.8 The runtime environment after `main` begins executing

After the call to `q`, an activation record for `q` has been added to the stack, the `ep` now points to the activation record of `q`, and `q` has stored the old `ep` as its control link. See Figure 10.9.

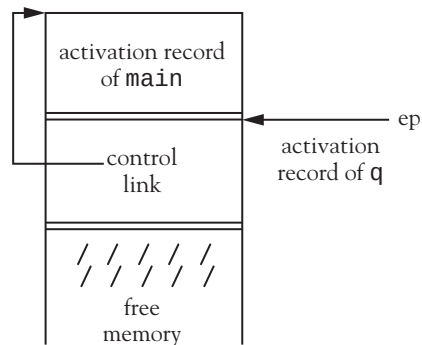


Figure 10.9 The runtime environment after `q` begins executing

When `p` is called inside `q`, a new frame is added for `p`. Thus, after the call to `p` from within `q`, the activation stack looks as shown in Figure 10.10.

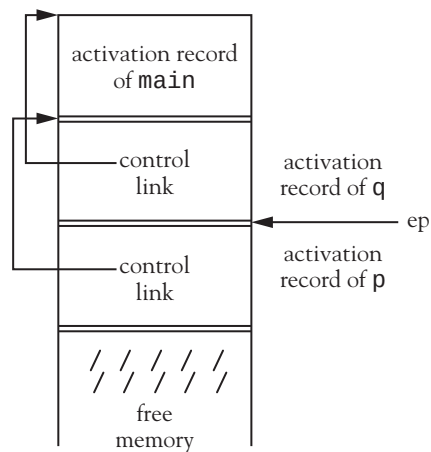


Figure 10.10 The runtime environment after `p` is called within `q`

Now when *p* is exited, the activation record of *p* is returned to free memory, and the *ep* is restored from the control link of the activation record of *p*, so that it points to the activation record of *q* again. Similarly, when *q* is exited, the *ep* is restored to point to the original environment of the main program.

With this new requirement, the fields in each activation record need to contain the information in Figure 10.11.

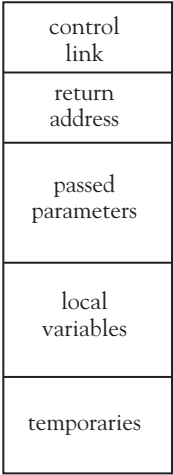


Figure 10.11 The component areas of an activation record with a control link

Consider now how the environment pointer can be used to find variable locations. Local variables are allocated in the current activation record, to which the *ep* points. Since the local variables are allocated as prescribed by the declarations of the block, and these declarations are static, each time the block is entered, the same kinds of variables are allocated in the same order. Thus, each variable can be allocated the same position in the activation record relative to the beginning of the record.¹¹ This position is called the **offset** of the local variable. Each local variable can be found using its fixed offset from the location to which the *ep* points.

Consider the following additions to our previous example:

```
int x;

void p( int y)
{ int i = x;
  char c;
  ...
}
```

(continues)

¹¹More precisely, recursive calls are permitted in Fortran77, but each new call overwrites the data of all previous calls, so recursion does not work. Procedures with this property are sometimes called **non-reentrant**.

(continued)

```
void q ( int a)
{ int x;
  ...
  p(1);
}

main()
{ q(2);
  return 0;
}
```

Any activation record of *p* will have a format like that in Figure 10.12. Each time *p* is called, the parameter *y* and the local variables *i* and *c* will be found in the same place relative to the beginning of the activation record.

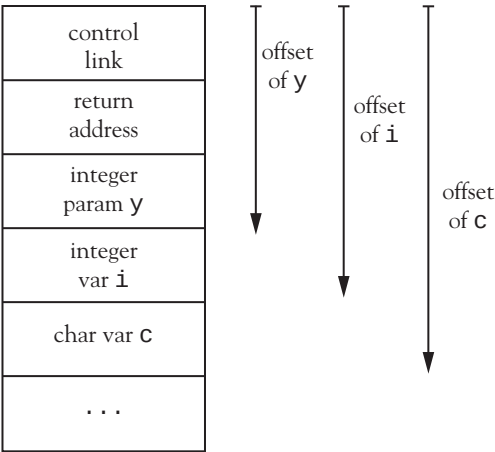


Figure 10.12 An activation record of *p* showing the offsets of the parameter and temporary variables

Now consider the case of nonlocal references, such as the reference to *x* in *p* indicated in the foregoing code. How is *x* found? Fortunately, in C as in FORTRAN, procedures cannot be nested, so all nonlocal references outside a procedure are actually global and are allocated statically. Thus, additional structures to represent closures are not required in the activation stack. Any nonlocal reference is found in the global area and can actually be resolved prior to execution.

However, many languages, including Pascal, Ada, and Modula-2, *do* permit nested procedures. With nested procedures, nonlocal references to local variables are permissible in a surrounding procedure scope. For example, see the following Ada code:

```

procedure q is
  x: integer;

  procedure p (y: integer) is
    i: integer := x;
  begin
    ...
  end p;

  procedure r is
    x: float;
  begin
    p(1);
    ...
  end r;

begin
  r;
end q;

```

Figure 10.13 shows what the activation stack looks like during the execution of p.

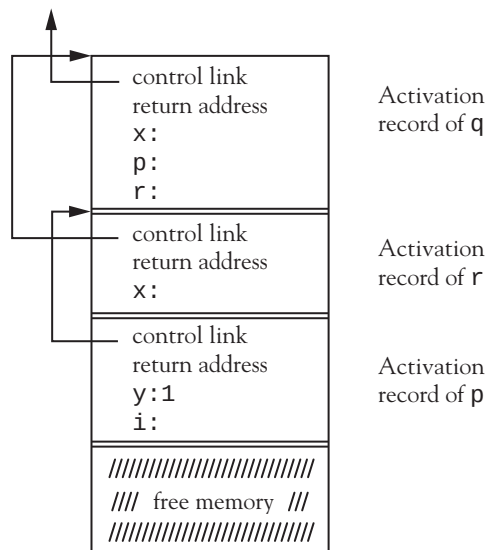


Figure 10.13 The runtime environment with activations of p, r, and q

How is it possible to find the nonlocal reference to the *x* of *q* from inside *p*? One idea would be to follow the control link to the activation record of *r*, but this would find the *x* local to *r*.¹² This would achieve dynamic scope rather than lexical scope. To achieve lexical scope, a procedure such as *p* must maintain a link to its **lexical** or **defining environment**. This link is called the **access link**, since it provides access to nonlocal variables. (Sometimes the access link is called the **static link**, since it is designed to implement lexical, or static, scope.) Since *p* is defined inside *q*, the access link of *p* is the *ep* that exists when *p* is defined, so the access link of *p* points to the activation of *q*. Now each activation record needs a new field, the access link field, and the complete picture of the environment for our example is as shown in Figure 10.14.

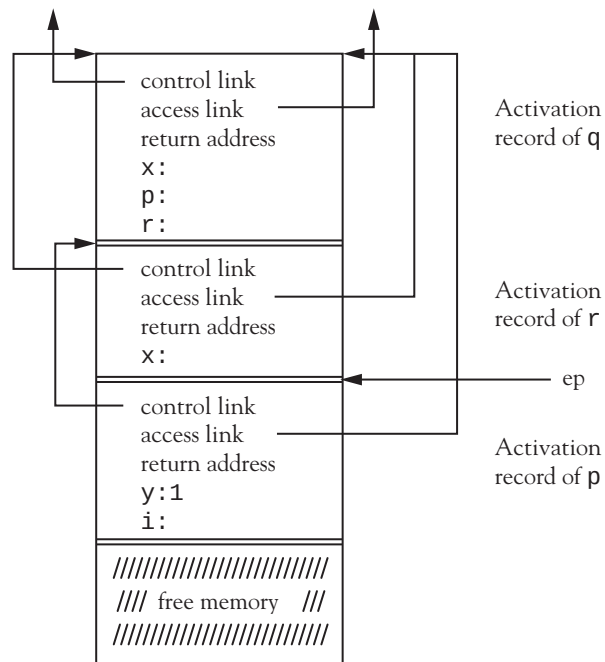


Figure 10.14 The runtime environment with activations of *p*, *r*, and *q* with access links

When blocks are deeply nested, it may be necessary to follow more than one access link to find a nonlocal reference. For example, in the Ada code

```

procedure ex is
  x: ... ;

  procedure p is
    ...
    procedure q is
      begin
        ... x ... ;

```

(continues)

¹²This would actually require that the full symbol table be maintained during execution, rather than replacing names by fixed offsets, since the offset (or even the existence) of a name in an arbitrary caller cannot be predicted to be at any fixed offset.

(continued)

```

        end q;
    begin -- p
        ...

    end p;
begin -- ex
    ...
end ex;

```

to access *x* from inside *q* requires following the access link in the activation record of *q* to its defining environment, which is an activation record of *p*, and then following the access link of *p* to the global environment. This process is called **access chaining**, and the number of access links that must be followed corresponds to the difference in nesting levels, or **nesting depth**, between the accessing environment and the defining environment of the variable being accessed.

With this organization of the environment, the closure of a procedure—the code for the procedure, together with a mechanism for resolving nonlocal references—becomes significantly more complex, since every time a procedure is called, the defining environment of that procedure must be included as part of the activation record. Thus, a function or procedure in a language like Ada or Pascal must be represented not just by a pointer to the code for the procedure but also by a closure consisting of a pair of pointers. The first of these is a code or instruction pointer, which we denote by *ip*. The second one is the access link, or environment pointer of its defining environment, which we denote by *ep*. We write this closure as $\langle ep, ip \rangle$, and we use it as the representation of procedures in the following discussion.

We finish this section with an example of an Ada program with nested procedures and a diagram of its environment at one point during execution (Figures 10.15 and 10.16). Note in the diagram of Figure 10.16 that the nested procedure *show* has two different closures, each corresponding to the two different activations of *p* in which *show* is defined.

```

(1) with Text_IO; use Text_IO;
(2) with Ada.Integer_Text_IO;
(3) use Ada.Integer_Text_IO;

(4) procedure lastex is

(5)   procedure p(n: integer) is

(6)     procedure show is
(7)       begin
(8)         if n > 0 then p(n - 1);

(9)       end if;

(10)      put(n);
(11)      new_line;
(12)    end show;

```

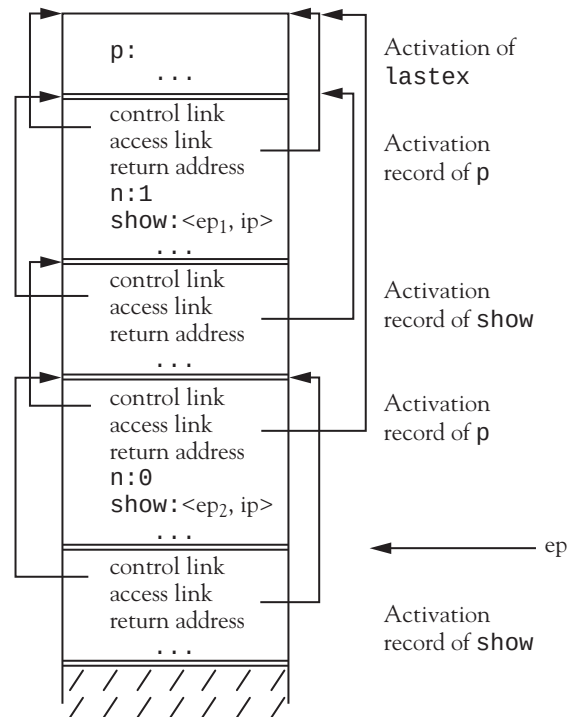
Figure 10.15 An Ada program with nested procedures (*continues*)

(continued)

```

(13)  begin -- p
(14)    show;
(15)  end p;
(16)  begin -- lastex
(17)    p(1);
(18)  end lastex;

```

Figure 10.15 An Ada program with nested procedures**Figure 10.16** Environment of *lastex* during the second call to *show* (line 8 of Figure 10.15)

10.4.3 Dynamically Computed Procedures and Fully Dynamic Environments

The stack-based runtime environment of the type shown in Figure 10.16 is completely adequate for almost all block-structured languages with lexical scope. The use of $\langle ep, ip \rangle$ closures for procedures makes this environment suitable even for languages with parameters that are themselves procedures, as long as these parameters are value parameters: A procedure that is passed to another procedure is passed as a closure (an $\langle ep, ip \rangle$ pair), and when it is called, its access link is the *ep* part of its closure. Examples of languages for which this organization is necessary are Ada and Pascal.

A stack-based environment does have its limitations, however, which we already noted in Chapter 8. For instance, any procedure that can return a pointer to a local object, either by returned value or through a pass by reference parameter, will result in a **dangling reference** when the procedure is exited, since the activation record of the procedure will be deallocated from the stack. The simplest example of this occurs when the address of a local variable is returned, as for instance in the C code:

```
int * dangle(void)
{ int x;
  return &x;
}
```

An assignment `addr = dangle()` now causes `addr` to point to an unsafe location in the activation stack.

This situation cannot happen in Java, however, since the address of a local variable is unavailable. Ada95 also makes this an error by stating the **Access-type Lifetime Rule**: An attribute `x'access` yielding a result belonging to an access type `T` (i.e., a pointer type) is only allowed if `x` can remain in existence at least as long as `T`. Thus, the Ada code equivalent to the above C code

```
type IntPtr is access Integer;

function dangle return IntPtr is
  x: Integer;
begin
  return x'access;
end dangle;
```

is illegal because the `IntPtr` type definition occurs in an outer scope relative to `x` (as it must to allow the definition of `dangle`), and so violates the Access-type Lifetime Rule.

However, there are situations in which a compiler cannot check for this error statically (see the Notes and References). In Ada an exception will still occur during execution (`Program_Error`), but in C/C++ and a few other languages this error will not be caught either statically or dynamically. In practice, it is easy for programmers who understand the environment to avoid this error.

A more serious situation occurs if the language designer wishes to extend the expressiveness and flexibility of the language by allowing procedures to be dynamically created. That is, if the language designer allows procedures to be returned from other procedures via returned value or reference parameters. This kind of flexibility is usually desired in a functional language, and even in some object-oriented languages, such as Smalltalk, where methods can be created dynamically. In a language where this is the case, procedures become first-class values, which means no “arbitrary” restrictions apply to their use. In such a language, a stack-based environment cannot be used, since the closure of a locally defined procedure will have an `ep` that points to the current activation record. If that closure is available outside the activation of the procedure that created it, the `ep` will point to

an activation record that no longer exists. Any subsequent call to that procedure will have an incorrect access environment.

Consider the following example, in which we may reasonably want a procedure to create another procedure. We write this example in Ada95, since Ada95 has procedure types and parameters, and so can express the situation quite well (this example is adapted from Abelson and Sussman [1996]):

```
type WithdrawProc is
  access function (x:integer) return integer;

InsufficientFunds: exception;

function makeNewBalance (initBalance: integer)
  return WithdrawProc
is
  currentBalance: integer;

  function withdraw (amt: integer) return integer is
  begin
    if amt <= currentBalance then
      currentBalance := currentBalance - amt;
    else
      raise InsufficientFunds;
    end if;
    return currentBalance;
  end withdraw;

begin
  currentBalance := initBalance;
  return withdraw'access;
end makeNewBalance;
```

We now might want to make two different accounts from which to withdraw, one with an initial balance of 500 and the other with an initial balance of 100 dollars:

```
withdraw1, withdraw2: WithdrawProc;

withdraw1 := makeNewBalance(500);
withdraw2 := makeNewBalance(100);
```

The problem is that in a stack-based environment, the activation records in which both these functions were created have disappeared: Each time `makeNewBalance` returns, the local environment of `makeNewBalance` is released. For example,

```
newBalance1 := withdraw1(100);
newBalance2 := withdraw2(50);
```

should result in a value of 400 for `newBalance1` and a value of 50 for `newBalance2`, but if the two instances of the local `currentBalance` variable (with values 500 and 100) have disappeared from the environment, these calls will not work.

Pascal does not have this problem, since there are no procedure variables (procedures can only be value parameters). In C all procedures are global, so again this problem cannot arise. Ada considers this program to be similar to the dangling reference code, and calls this a violation of the Access-type Lifetime Rule. Thus, Ada will generate a compile-time error message in this case.

Nevertheless, we may want to be able to do things just like this in a language in which functions and procedures are first class values; that is, no nongeneralities or nonorthogonalities should exist for functions and procedures. Such a language, for example, is LISP. What kind of environment would we need to allow such constructs?

Now it is no longer possible for the activation record of procedure `makeNewBalance` to be removed from the environment, as long as there are references to any of its local objects. Such an environment is **fully dynamic** in that it deletes activation records only when they can no longer be reached from within the executing program. Such an environment must perform some kind of automatic reclamation of unreachable storage. Two standard methods of doing so are **reference counts** and **garbage collection**; these will be studied in Section 10.5.

This situation also means that the structure of the activations becomes treelike instead of stacklike. The control links to the calling environment no longer necessarily point to the immediately preceding activation. For example, in the two calls to `makeNewBalance`, the two activations remain, with their control links both pointing at the defining environment. See Figure 10.17.

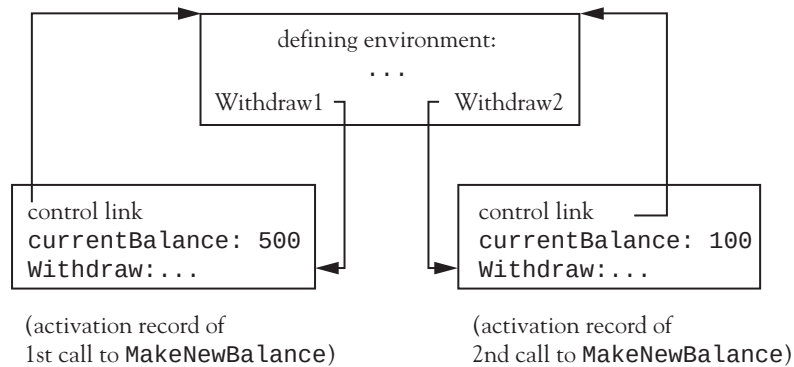


Figure 10.17 A tree-like runtime environment for a language with first-class procedures

Each activation of `makeNewBalance` can disappear only if `withdraw1` or `withdraw2` are reassigned or disappear themselves.

This is the model under which Scheme and other functional languages execute. In the next section, we will discuss issues involved in maintaining such environments.

10.5 Dynamic Memory Management

In a typical imperative language such as C, the automatic allocation and deallocation of storage occurs only for activation records on a stack. In this relatively easy process, space is allocated for an activation record when a procedure is called and deallocated when the procedure is exited. Explicit dynamic allocation and use of pointers is also available under manual programmer control using a **heap** of memory separate from the stack, as we described in Chapter 7. However, as we noted there, manual memory management on the heap suffers from a number of potential problems, including the creation of garbage and dangling references. Thus, languages with significant needs for heap storage, such as Java, are better off leaving nonstack dynamic storage to a memory manager that provides automatic garbage collection.

This means that any language that does not apply significant restrictions to the use of procedures and functions *must* provide automatic garbage collection, since the stack-based system of procedure call and return is no longer correct. Functional languages such as Scheme, ML, and Haskell, which have first-class function values, fall into this category, as do object-oriented languages such as Smalltalk and Java.

A very simple solution to the problem of managing dynamic memory is to *not* deallocate memory once it has been allocated. In other words, every call to a function creates a new activation record in memory that is not deallocated on exit. This method has two advantages: It is correct, and it is easy to implement. What's more, it actually can work for small programs. However, it is not really an option for most programs written in functional and object-oriented languages. After all, in these languages, processing occurs primarily by function or method call, many instances of which may be recursive. Moreover, the internal representation of data uses pointers and requires a substantial amount of indirection and memory overhead. Thus, if deallocation does not occur, memory is very quickly exhausted.

However, this method *has* been used in conjunction with virtual memory systems, since the address space for such systems is almost inexhaustible. This only defers the basic problem, however, since it causes the swap space of the operating system to become exhausted and can cause a serious degradation of performance because of page faults. These topics are beyond the scope of this book but at least serve to point out interplay between memory management and operating system issues.

Automatic memory management actually falls into two categories: the **reclamation** of previously allocated but no longer used storage, previously called **garbage collection**, and the **maintenance** of the free space available for allocation. Indeed, maintenance of free space is needed even for manual heap operations in languages such as C; it is also a little more straightforward than is reclamation, so we discuss it first.

10.5.1 Maintaining Free Space

Generally, a contiguous block of memory is provided by the operating system for the use of an executing program. The free space within this block is maintained by a list of free blocks. One way to do this is via a linked list, such as the circular list in the following figure, indicating the total available space, with allocated blocks shaded and free blocks blank. See Figure 10.18.

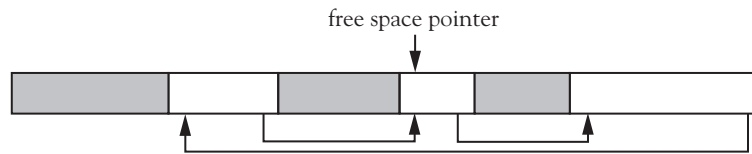


Figure 10.18 The free space represented as a linked list

When a block of a certain size needs to be allocated, the memory manager searches the list for a free block with enough space, and then adjusts the free list to remove the allocated space, as in the picture shown in Figure 10.19.

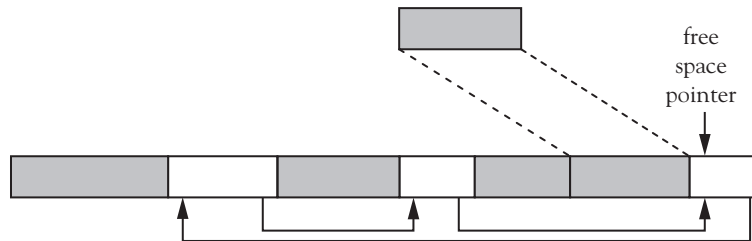


Figure 10.19 Allocating storage from the free space

When memory is reclaimed, blocks are returned to the free list. For example, if the light-shaded areas are storage to be reclaimed (as shown in Figure 10.20), then the new free list after reclamation would look as shown in Figure 10.21.

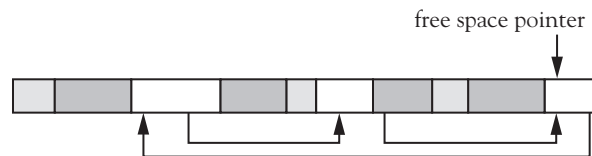


Figure 10.20 Returning storage to the free list

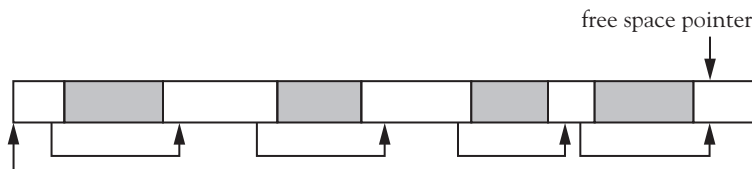


Figure 10.21 The free list after the return of storage

When blocks of memory are returned to the free list, they must be joined with immediately adjacent blocks to form the largest contiguous block of free memory. This process is called **coalescing**. In the preceding example, the small block freed in the middle of the memory area was coalesced with the free block adjacent to it on the right. Even with coalescing, however, a free list can become **fragmented**—that is, it can break into a number of small-sized blocks. When this occurs, it is possible for the allocation of a large block to fail, even though there is enough total space available to allocate it. To prevent this, memory must occasionally be **compacted** by moving all free blocks together and coalescing them into

one block. For example, the five blocks of free memory in the previous diagram could be compacted as shown in Figure 10.22.

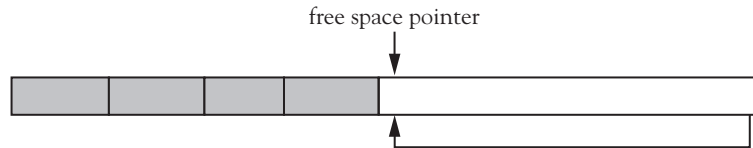


Figure 10.22 The free list after compaction

Storage compaction involves considerable overhead, since the locations of most of the allocated quantities will change and data structures and tables in the runtime environment will have to be modified to reflect these new locations.

10.5.2 Reclamation of Storage

Recognizing when a block of storage is no longer referenced, either directly or indirectly through pointers, is a much more difficult task than the maintenance of the free list itself. Historically, two main methods have been used: reference counting and mark and sweep.

Reference counting is considered an “eager” method of storage reclamation, in that it tries to reclaim space as soon as it is no longer referenced. Each block of allocated storage contains an extra count field, which stores the number of references to the block from other blocks. Each time a reference is changed, these reference counts must be updated. When the reference count drops to zero, the block can be returned to the free list. In theory this seems like a simple and elegant solution to the problem, but in fact it suffers from serious drawbacks. One obvious one is the extra memory that it takes to keep the reference counts themselves. Even more serious, the effort to maintain the counts can be fairly large. For example, when making an assignment, which we will model by a C-like pointer assignment $p = q$, first the old value of p must be followed, and its reference count decremented by one. If this reference count should happen to drop to zero, it is not sufficient simply to return it to free storage, since it may itself contain references to other storage blocks. Thus, reference counts must be decremented recursively. Finally, when q is copied to p , its reference count must be incremented. A pseudocode description of assignment would, therefore, look as follows:

```
void decrement(p)
{ p->refcount--;
  if (p->refcount == 0)
  { for all fields r of *p that are pointers do
    decrement(r);
    deallocate(*p);
  }
}
void assign(p,q)
{ decrement(p);
  p = q;
  q->refcount++;
}
```

However, the overhead to maintain reference counts is not the worst flaw of this scheme. Even more serious is that circular references can cause unreferenced memory to never be deallocated.

For example, consider a circular linked list such as the one shown in Figure 10.23.

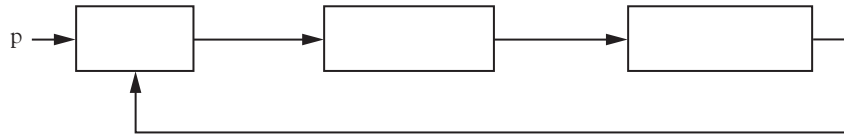


Figure 10.23 A circular linked list

If *p* is deallocated, the reference count of the block to which *p* points will drop from 2 to 1. The entire list will never be deallocated.

The standard alternative to reference counts is **mark and sweep**. Whereas reference counting is considered an eager method, mark and sweep is a “lazy” method, in that it puts off reclaiming any storage until the allocator runs out of space, at which point it looks for all storage that can be referenced and moves all unreferenced storage back to the free list. It does this in two passes. The first pass follows all pointers recursively, starting with the current environment or symbol table, and marks each block of storage reached. This process requires an extra bit of storage for the marking. A second pass then sweeps linearly through memory, returning unmarked blocks to the free list.

In method, unlike with reference counts, freeing blocks with circular references is not a problem. However, mark and sweep requires extra storage. It also suffers from another serious drawback: The double pass through memory causes a significant delay in processing, sometimes as much as a few seconds, each time the garbage collector is invoked, which can be every few minutes. This is clearly unacceptable for many applications involving interactive or immediate response. However, many modern LISP and Smalltalk implementations are still plagued by this issue.

One way around the problem is to split available memory into two halves and allocate storage only from one-half at a time. Then, during the marking pass, all reached blocks are immediately copied to the second half of storage not in use. In this method, which is often called **stop and copy**, no extra mark bit is required in storage, and only one pass is required. It also performs compaction automatically. Once all reachable blocks in the used area have been copied, the used and unused halves of memory are interchanged, and processing continues. However, this does little to improve processing delays during storage reclamation.

In the 1980s, a method was invented that reduces this delay significantly. Called **generational garbage collection**, it adds a permanent storage area to the reclamation scheme of the previous paragraph. Allocated objects that survive long enough are simply copied into permanent space and are never deallocated during subsequent storage reclamations. This means that the garbage collector needs to search only a very small section of memory for newer storage allocations, and the time for such a search is reduced to a fraction of a second. Of course, it is possible for permanent memory still to become exhausted with unreachable storage, but this is a much less severe problem than before, since temporary storage tends to disappear quickly, while storage that stays allocated for some time is small and tends to persist anyway. This process has been shown to work very well, especially with a virtual memory system.

10.6 Exception Handling and Environments

In the previous chapter, we described exception handling in some detail in terms of its syntax, semantics, and use. This section introduces some implementation issues, particularly as they affect the structure and behavior of the runtime environment. Further detail can be found in the sources listed in the Notes and References.

In principle, the operations of raising and handling exceptions are similar to procedure calls, and they can be implemented in similar ways. However, there are major differences as well. Primary among these are:

1. An activation cannot be created on the runtime stack to represent the raising of an exception, since this stack itself may be unwound as previously described when searching for a handler.
2. A handler must be found and called dynamically, rather than statically as with ordinary function calls.
3. The actions of a handler are based on the type of the exception, rather than the exception value itself; thus, exception type information must be retained during execution, contrary to standard practice for statically typed languages.¹³

The first difference can be easily overcome. When an exception is raised, no activation record is created. Instead, the exception object and its type information are placed in a known location, such as a register or static memory. A jump is then performed to generic code that looks for a handler. Exit code is called if a handler is not found. The return address for the successful handling of an exception must also be stored in a known location. Under the termination model, this address is the location following the block in which the exception occurred. Otherwise, it is the return address of the most recent call, if the block is a procedure.

The second difference is more problematic. Pointers to handlers must, at least in theory, be kept on some kind of stack. Each time code is entered that has an associated handler, a new handler pointer is pushed, and when that same code is exited, the pointer is popped again to reveal any previous handlers. If this stack is to be implemented directly, it must be maintained either on the heap or elsewhere in its own memory area (i.e., not on the runtime stack), and a pointer to the current stack top must be maintained, either in static memory or in a register.

The third difference is somewhat less problematic but may involve some complexity that we will not go into here. The main issue is how to record the needed type information (essentially the type names) without added overhead in the exception structures themselves. One possibility is to have some sort of lookup table.

Once these issues have been addressed, the implementation of handlers is relatively straightforward. The basic idea is to collect all handler code attached to a particular block into a single handler implemented as a switch statement, based on the type of the exception parameter received, with a

¹³Of course, languages with dynamic types, such as LISP and Smalltalk, already have type information available at runtime. C++ and Java also have mechanisms available for preserving and extracting type information when necessary at runtime; in C++ the mechanism is called RTTI (for Run Time Type Information); in Java it is called Reflection.

default case that pops the handler stack (adjusting the current return address) and, if necessary, pops the runtime stack before reraising the same exception. (Procedure blocks must at least have this latter default handler, even if no exceptions are explicitly handled.) For example, consider the following handler code:

```
void factor() throw (Unwind,InputError)
{ try
  {...}
  catch (UnexpectedChar u)
  {...}
  catch (Unwind u)
  {...}
  catch (NumberExpected n)
  {...}
}
```

The handler for this try block might be implemented as in the following pseudocode:

```
switch (exception.type)
{ case UnexpectedChar: ...; break;
  case Unwind : ...; break;
  case NumberExpected: ... ; break;
  default: pop the runtime stack and reraise the exception;
}
pop the runtime stack and return to caller;
```

The main problem with the implementation techniques described so far is that the maintenance of the handler stack causes a potentially significant runtime penalty, even for code that does not use exception handling. One would like to provide an alternative to the handler stack that can be statically generated and, thus, cost nothing for code that does not use exception handling (this was a major design goal for C++). Such an alternative is a sorted table of code addresses that records the available handlers. When an exception occurs, the exception code performs a lookup based on the code address where the exception occurred (a binary search of the address table, for example). If no handler is found, the block in which the exception occurred is exited and the exit address is used for a new lookup.

Of course, such an address table has its own problems. First, the table itself can be quite large, causing the memory use of the program to grow significantly (a classic time-space tradeoff). Second, when an exception does occur, there can be an even greater execution speed penalty because of multiple searches of the address table. In a typical C++ system, for example, handling an exception may be as much as two orders of magnitude slower than a simple jump such as a loop exit. Thus, it makes even more sense in such systems to avoid using exceptions for routine control flow, and save them for truly unusual events.

10.7 Case Study: Processing Parameter Modes in TinyAda

Being a parameter is one of the roles that an identifier can play in TinyAda. In the role analysis discussed in Section 7.8.2, the use of parameters was subjected to the same restrictions as the use of variables. They could appear in any expression or on the left side of an assignment statement. In Section 9.6, we saw that neither variables nor parameters can appear in static expressions. In the present chapter, we have seen how parameters can be further specified in terms of the roles they play in conveying information to and from procedures. These new roles are called parameter modes, and they allow us to apply a new set of constraints during semantic analysis, to which we now turn.

10.7.1 The Syntax and Semantics of Parameter Modes

The syntax for declaring TinyAda’s parameter modes is specified in the two syntax rules for parameter specifications:

```
parameterSpecification = identifierList ":" mode <type>name

mode = [ "in" ] | "in" "out" | "out"
```

Parameter modes are marked by the single reserved words `in` and `out`, or by these two words in succession, or by no marker at all. The last option is a default, which has the same meaning as `in` only.

As discussed in Section 10.3.5, a parameter declared with the mode of `in` is intended for input to a procedure only. As such, it can appear in any expression to be referenced for its value, but it cannot be the target of an assignment statement. Also, an `in` parameter cannot be passed as an actual parameter to a procedure that is expecting an `out` or an `in out` parameter at that position in its formal parameter list. Inversely, an `out` parameter is intended for output from a procedure only, so it cannot be referenced for its value. Thus, an `out` parameter can only be the target of an assignment statement, or be passed as an actual parameter to a procedure that expects another `out` parameter in that position in its formal parameter list. Finally, an `in out` parameter can be used for both input and output in a procedure. Thus, it can appear anywhere in the body of a procedure that a variable can. Table 10.1 summarizes the static semantics of TinyAda’s parameter modes.

Table 10.1 The static semantics of TinyAda’s parameter modes		
Parameter Mode	Role in a Procedure	Semantic Restrictions
<code>in</code>	Input only	May appear only in expressions, or only in the position of an <code>in</code> parameter when passed as an actual parameter to a procedure.
<code>out</code>	Output only	May appear only as the target of an assignment statement, or only in the position of an <code>out</code> parameter when passed as an actual parameter to a procedure.
<code>in out</code>	Input and output	May appear anywhere that a variable may appear.

10.7.2 Processing Parameter Declarations

To store information about a parameter’s mode, the `SymbolEntry` class is modified to include a `mode` field, with the possible values `SymbolEntry.IN`, `SymbolEntry.OUT`, and `SymbolEntry.IN_OUT`. The method `setMode` is also provided to install a parameter mode into a list of parameter identifiers. Armed

with these changes, we can now recast the parsing method for `parameterSpecification` to recognize and enter parameter mode information, as follows:

```
/*
parameterSpecification = identifierList ":" mode <type>name
*/
private SymbolEntry parameterSpecification(){
    SymbolEntry list = identifierList();
    list.setRole(SymbolEntry.PARAM);
    accept(Token.COLON, ":" expected);
    if (token.code == Token.IN){
        token = scanner.nextToken();
        list.setMode(SymbolEntry.IN);
    }
    if (token.code == Token.OUT){
        token = scanner.nextToken();
        if (list.mode == SymbolEntry.IN)
            list.setMode(SymbolEntry.IN_OUT);
        else
            list.setMode(SymbolEntry.OUT);
    }
    if (list.mode == SymbolEntry.NONE)
        list.setMode(SymbolEntry.IN);
    SymbolEntry entry = findId();
    list.setType(entry.type);
    acceptRole(entry, SymbolEntry.TYPE, "type name expected");
    return list;
}
```

Note that the default value of the mode in the symbol entry is `SymbolEntry.NONE`, which will be the case if the parser does not see any of the explicit reserved words for modes in the token stream. In that case, the mode is reset to `SymbolEntry.IN`.

10.7.3 Processing Parameter References

Table 10.1 provides the guidelines for checking references to parameters in a TinyAda source program. First, let's consider parameter names that appear in expressions and assignment statements. A name in an expression is detected in the method `primary`. If that name is a parameter, then its mode cannot be `OUT`. Otherwise, the parameter name is accepted. A name that is the target of an assignment statement is detected in the method `assignmentOrCallStatement`. If that name is a parameter, then its mode cannot be `IN`. Otherwise, the parameter name is accepted.

The only other location where parameter modes must be checked is during the parsing of actual parameters to procedure calls. The method `actualParameterPart` was developed in Section 8.10.5 to match the number and types of actual parameters against those of the procedure's formal parameters. This method must now also examine the modes of the formal parameters in order to apply the new constraints listed in

Table 10.1. If the mode of the formal parameter is IN, then the method expression is called, as before, to parse the actual parameter. Otherwise, the mode of the formal parameter must be OUT or IN_OUT, so the method name is called to parse the actual parameter. Then, the name just processed must be a variable or a parameter. If the name is a parameter, its mode cannot be IN. Otherwise, the parameter name is accepted.

The modification of these parsing methods is left as an exercise for the reader.

Exercises

- 10.1 Beginning programmers often add a return statement at the end of each procedure:

```
void inc (int* x)
{ (*x)++;
  return;
}
```

The thinking is that, as with switch statements, there might be a “fall-through” if the return is not there. Could a function mechanism be designed that would allow this? Would it make sense to have such a mechanism?

- 10.2 What if we wrote an increment procedure in C as follows (removing the parentheses around *x):

```
void inc (int* x)
{ *x++;
}
```

Is this legal? If so, what does this procedure do? If not, why not?

- 10.3 Can a swap procedure that interchanges the values of its parameters be written in Java? Discuss.

- 10.4 (a) Consider the intswap C++ procedure of Section 10.1:

```
void intswap (int& x, int& y)
{ int t = x;
  x = y;
  y = t;
}
```

Is this procedure in closed form? Why or why not?

- (b) Is the following polymorphic swap procedure in C++ in closed form (why or why not)?

```
template <typename T>
void swap (T& x, T& y)
{ T temp = x;
  x = y;
  y = temp;
}
```

- 10.5 Suppose we tried to write a C procedure that would initialize an array to a certain size as follows:

```
void init(int a[], int size)
{ a = (int*) malloc(size*sizeof(int));
}
```

What, if anything, is wrong with this? Explain.

- 10.6 Here are some function declarations in C:

```
void f(int x, double y);
void f(int a, double);
void f(int, double z);
void f(int y, double q)
{ if (q > y) printf("ok!\n");
  else printf("not ok!\n");
}
```

Would it be legal to include all of the declarations in the same code file? Explain.

- 10.7 Suppose that we tried to write a procedure in Java that would append “hello” to the end of a string:

```
class TestParams
{ public static void appendHello( String s)
  { s = s+"hello";
  }
  public static void main( String[] args)
  { String s = "Greetings! ";
    appendHello(s);
    System.out.println(s);
  }
}
```

Explain the behavior of this program. Can it be fixed?

- 10.8 Give the output of the following program (written in C syntax) using the four parameter-passing methods discussed in Section 10.3:

```
int i;
int a[2];

void p( int x, int y)
{ x++;
  i++;
  y++;
}
main()
{ a[0] = 1;
```

(continues)

(continued)

```

        a[1] = 1;
        i = 0;
        p(a[i],a[i]);
        printf("%d\n",a[0]);
        printf("%d\n",a[1]);
        return 0;
    }

```

10.9 Give the output of the following program using the four parameter-passing methods of Section 10.3:

```

int i;
int a[3];

void swap( int x, int y)
{ x = x + y;
  y = x - y;
  x = x - y;
}

main()
{ i = 1;
  a[0] = 2;
  a[1] = 1;
  a[2] = 0;
  swap(i,a[i]);
  printf("%d %d %d %d\n", i, a[0], a[1], a[2]);
  swap(a[i],a[i]);
  printf("%d %d %d\n", a[0], a[1], a[2]);
  return 0;
}

```

10.10 FORTRAN has the convention that all parameter passing is by reference. Nevertheless, as described in the chapter, it is possible to call subroutines with nonvariable expressions as arguments, as in `CALL P(X,X+Y,2)`.

- (a) Explain how one can pass a variable `X` by value in FORTRAN.
- (b) Suppose that subroutine `P` is declared as follows:

```

SUBROUTINE P(A)
INTEGER A
PRINT *, A
A = A + 1
RETURN
END

```

and is called from the main program as follows:

```
CALL P(1)
```

In some FORTRAN systems, this will cause a runtime error. In others, no runtime error occurs, but if the subroutine is called again with 1 as its argument, it may print the value 2.

Explain how both behaviors might occur.

- 10.11 In Ada the compiler checks to make sure that the value of an `in` parameter is not changed in a procedure, and then allows `in` parameters to be implemented using pass by reference. Why not insist on pass by value for `in` parameters, so that the compiler check can be eliminated (as in C and Java)?
- 10.12 Ada restricts the parameters in a function declaration to be `in` parameters. Why is this?
- 10.13 A variation on pass by name is **pass by text**, in which the arguments are evaluated in delayed fashion, just as in pass by name, but each argument is evaluated in the environment of the called procedure, rather than in the calling environment. Show that pass by text can have different results from pass by name.
- 10.14 The Algol60 definition states the following **substitution rule** for pass by name:
 1. Any formal parameter ... is replaced, throughout the procedure body, by the corresponding actual parameter, after enclosing this latter in parentheses wherever syntactically possible. Possible conflicts between identifiers inserted through this process and other identifiers already present within the procedure body will be avoided by suitable systematic changes of the formal or local identifiers involved.
 2. Finally the procedure body, modified as above, is inserted in place of the procedure statement and executed. If the procedure is called from a place outside the scope of any nonlocal quantity of the procedure body the conflicts between the identifiers inserted through this process of body replacement and the identifiers whose declarations are valid at the place of the procedure statement or function designator will be avoided through suitable systematic changes of the latter identifiers (Naur [1963a], p. 12).
 - (a) Give an example of an identifier conflict as described in rule 1.
 - (b) Give an example of an identifier conflict as described in rule 2.
 - (c) Carry out the replacement process described in these two rules on the program in Figure 10.4.
- 10.15 The following two exercises relate to Jensen's device and pass by name, as discussed in Section 10.3.4.
 - (a) Write a function that uses Jensen's device and pass by name to compute the scalar product of two vectors declared as integer arrays with the same number of elements.
 - (b) The following sum procedure (in C syntax) was used as an example of pass by name and Jensen's device at the end of Section 10.3.4:

```
int sum (int a, int index, int size)
{ int temp = 0;
  for (index = 0; index < size; index++) temp += a;
  return temp;
}
```

Rewrite this code using a function parameter for the parameter `a` to imitate pass by name, so that it actually executes like pass by name in C.

- 10.16 The text did not discuss the scope of parameter declarations inside function or procedure declarations.

(a) Describe the scope of the declaration of *x* in the following C procedure:

```
void p( int x)
{  int y, z;
    ...
}
```

(b) State whether the following C procedure declarations are legal, and give reasons:

```
void p( int p) { ... }
void q( int x) { int x; ... }
```

- 10.17 Draw the stack of activation records for the following C program after (a) the call to *r* on line 13 and (b) after the call to *p* on line 14. Show the control links, and fill in the local names in each activation record. (c) Describe how variables *r* and *x* are found on lines 4 and 5 during the execution of *p*.

```
(1)  int x;
(2)  void p(void)
(3)  { double r = 2;
(4)    printf("%g\n", r);
(5)    printf("%d\n", x);
(6)  }
(7)  void r(void)
(8)  { x = 1;
(9)    p();
(10) }
(11) void q(void)
(12) { double x = 3;
(13)  r();
(14)  p();
(15) }
(16) main()
(17) { p();
(18)  q();
(19)  return 0;
(20) }
```

- 10.18 Draw the stack of activation records for the following Ada program **(a)** after the first call to procedure b; **(b)** after the second call to procedure b. Show the control links *and* the access links for each activation record. **(c)** Indicate how b is found in procedure c.

```

procedure env is

  procedure a is

    procedure b is

      procedure c is
      begin
        b;
      end c;

    begin
      c;
    end b;

  begin
    b;
  end a;

begin
  a;
end env;

```

- 10.19 The following Ada program contains a function parameter.
- (a)** Draw the stack of activation records after the call to g in p. **(b)** What does the program print and why?

```

with Text_IO; use Text_IO;
with Ada.Integer_Text_IO;
use Ada.Integer_Text_IO;

procedure params is
  procedure q is
    type IntFunc is access function (n:integer) return integer;
    m: integer := 0;

    function f (n: integer) return integer is
    begin
      return m + n;
    end f;
  procedure p (g: IntFunc) is

```

(continues)

(continued)

```

        m: integer := 3;
    begin
        put(g(2)); new_line;
    end p;

    begin
        p(f' access);
    end q;

begin
    q;
end params;

```

- 10.20 Show that the access link in an activation record (the “static” link) isn’t static.
- 10.21 Suppose that we tried to avoid adding an access link to each activation record in a language with nested procedures by devising a mechanism for counting our way through dynamic links until the appropriate activation was reached (dynamic links will always lead eventually to the correct lexical environment). Why can’t this work?
- 10.22 FORTRAN allows the passing of variable-length arrays, as in the following examples:

```

SUBROUTINE P(A, I)
  INTEGER I
  REAL A(*)
  ...
  SUBROUTINE Q(B, N)
    INTEGER N, B(N)
    ...

```

Does this cause any problems for the activation records of P and Q, which in FORTRAN must have fixed size and location?

- 10.23 Some FORTRAN implementations use pass by value-result rather than pass by reference. Would this affect your answer to the previous exercise? Why?
- 10.24 In C, blocks that are not procedures, but have variable declarations, such as:

```

{ int i;
  for (i=1; i<n; i++) a[i] = i;
}

```

can be allocated an activation record just as procedures. What fields in the activation record are unused or redundant in this case? Can such blocks be implemented without a new activation record?

- 10.25 Suppose that we disallowed recursion in Ada. Would it be possible to construct a fully static environment for the language? Is the same true for C?

- 10.26 Describe how, in a language without procedure parameters, the environment pointer of a procedure closure does not need to be stored with the procedure, but can be computed when the procedure is called.
- 10.27 Suppose that we wanted to state a rule that would make the C dangling reference created by the following function illegal:

```
int* dangle(void)
{ int x;
  return &x;
}
```

Suppose that we decided to make the following statement: The address of a local variable cannot be a returned value and cannot be assigned to a nonlocal variable.

- (a) Can this rule be checked statically?
- (b) Does this rule solve the problem of dangling references created by the use of a stack-based environment?
- 10.28 The example program `params` of Exercise 10.19 remains valid from the point of view of a stack-based runtime environment if the definitions of procedure `g` and the `IntFunc` type are moved outside of procedure `q`, and yet Ada considers the new program to be erroneous.
- (a) Draw the new runtime environment after the call to `g` in `p` that results from this change to show that it is still valid.
- (b) Discuss the reasons that Ada declares an error in this program (see the access-type lifetime rule discussion in Section 10.4.3).
- 10.29 The following program in Ada syntax has a function that has another function as its returned value, and so needs a fully dynamic runtime environment as described in Section 10.4.3. Draw a picture of the environment when `g` is called inside `b`. What should this program print?

```
with Text_IO; use Text_IO;
with Ada.Integer_Text_IO;
use Ada.Integer_Text_IO;

procedure ret is
  type IntFunc is
    access function (n:integer) return integer;

  function a return IntFunc is
    m: integer;

    function addm( n: integer) return integer is
    begin
      return (n + m);
    end addm;
```

(continues)

(continued)

```

begin
  m := 0;
  return addm'access;
end a;

procedure b(g: IntFunc) is
begin
  put(g(2));
end b;

begin
  b(a);
end ret;
```

- 10.30 Using a debugger, try to determine whether your C++ compiler uses static address tables to look up exception handlers. Describe what the debugger tells you is occurring when an exception is raised.
- 10.31 Write declarations in a language of your choice for the maintenance of a free list of memory as **(a)** a circular singly linked list, and **(b)** a noncircular doubly linked list. Write procedures to deallocate a block using both of these data structures, being sure to coalesce adjacent blocks. Compare the ease of coalescing blocks in each case.
- 10.32 Show how compaction of memory can be accomplished by using a table to maintain an extra level of indirection, so that when a block of storage is moved, only one pointer needs to be changed.
- 10.33 In Section 10.5, a method of mark and sweep garbage collection was described in which memory is split into two sections, only one of which is used for allocation between calls to the garbage collector. It was claimed in the text that this eliminates the need for extra space to mark memory that has been seen. Describe how this can be accomplished.
- 10.34 In the generational method of garbage collection, it is still necessary eventually to reclaim “permanently” allocated storage after long execution times. It has been suggested that this could be made to interfere the least with execution by scheduling it to be performed “offline,” that is, during periods when the running program is waiting for input or other processing. For this to be effective, the garbage collector must have a good interface with the operating system, especially if virtual memory is in use. Describe as many operating system issues as you can think of in this approach.
- 10.35 Modify the method `assignmentOrCallStatement` in the `TinyAda` parser so that it enforces the constraints on parameter names that are the targets of assignment statements.
- 10.36 Modify the method `primary` in the `TinyAda` parser so that it enforces the constraints on parameter names that appear in expressions.
- 10.37 Modify the method `actualParameterPart` in the `TinyAda` parser so that it enforces the constraints on actual parameters that are passed as arguments to procedures.

Notes and References

Structures of runtime environments are described in more detail in Louden [1997] and Aho, Sethi, and Ullman [1986]. Closures of procedures with nonlocal references are related to closures of lambda expressions with free variables; see Section 10.7. The access-type lifetime rule in Ada is discussed in more detail in Cohen [1996].

Dynamic memory management is treated in more detail in Horowitz and Sahni [1984] and Aho, Hopcroft, and Ullman [1983]. Traditionally, languages that have first-class procedures and, thus, must use automatic memory management, have been thought to be highly inefficient for that reason. Advances in algorithms, such as generational garbage collection, and advances in translation techniques make this much less true. References dealing with efficiency issues are Steele [1977] and Gabriel [1985]; see also Louden [1987]. Overviews of garbage collection techniques can be found in Wilson [1992] or Cohen [1981]. Generational garbage collection was first described in Ungar [1984]. A generational garbage collector and runtime environment for the functional language ML is described in Appel [1992].

Exception handling implementation is discussed in Scott [2000] and Stroustrup [1994]. Details of two implementations can be found in Koenig and Stroustrup [1990] and Lenkov et al. [1992]. See also Lajoie [1994ab].